

Rapport de Projet: People Detection

Realisé par:

Hassan Amrani
Wassim Boutrasseyt
Aomari Ayman
Walid hbabou

Encadré par:

Mr Zaati

Sommaire

- 01. Introduction
- 02. Technologies
- 03. Établissement de la Connexion Serveur-Client
- 04. Acquisition et Prétraitement des Images
- 05. Détection et Traitement des Données
- 06. Intégration Cloud -Azure-
- 07. Application
- 08. Conclusion

Introduction

Avec l'essor des technologies d'intelligence artificielle et de cloud computing, la surveillance et la détection de personnes dans les espaces publics ont connu une avancée significative. Ce projet vise à développer un système automatisé de détection de personnes en temps réel, en utilisant des caméras publiques et des techniques de traitement d'image avec OpenCV. Les informations capturées, telles que les coordonnées et les heures de détection des individus, sont ensuite stockées dans un fichier CSV pour une analyse approfondie et une traçabilité efficace, en utilisant Matplotlib pour visualiser les résultats de manière graphique.

L'intégration avec le cloud Azure permet de tirer parti de sa puissance de calcul et de ses services de stockage pour traiter et conserver les données en toute sécurité, même à grande échelle. Azure facilite également le déploiement de notre solution dans des environnements distribués, garantissant ainsi une disponibilité et une résilience optimales pour notre système de détection.

Ce projet se compose de quatre phases principales :

1. *Établissement de la Connexion Serveur-Client* : La première phase consiste à mettre en place une connexion réseau via des sockets pour assurer la communication entre le serveur central et les différents dispositifs (caméras publiques). Cette phase permettra de capter en temps réel les flux vidéo transmis par les caméras vers le serveur pour le traitement.
2. *Acquisition et Prétraitement des Images* : Utilisation d'une caméra publique pour capturer des flux vidéo en temps réel, avec le traitement d'image effectué via OpenCV. Cette étape inclut la gestion des flux vidéo reçus et la préparation des images pour la détection.
3. *Détection et Traitement des Données* : Identification des personnes dans le flux vidéo par l'application de méthodes de détection d'objets sur les images capturées. OpenCV sera utilisé pour traiter chaque image et extraire les informations pertinentes sur la présence des individus.
4. *Stockage, Visualisation et Intégration Cloud* : Stockage des informations de détection dans des fichiers CSV pour une analyse ultérieure, et visualisation des résultats avec Matplotlib. Les données sont ensuite intégrées au cloud Azure pour leur gestion et stockage à grande échelle.

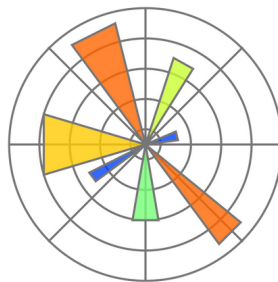
En combinant la puissance du traitement d'images avec OpenCV, la visualisation des données via Matplotlib, et les services cloud d'Azure, ce projet constitue une solution innovante et scalable pour la surveillance automatisée. Il présente des applications potentielles dans des domaines variés tels que la sécurité, l'analyse du trafic piéton et bien d'autres.

Technologies

Python est un langage de programmation interprété, de haut niveau, connu pour sa syntaxe simple et lisible. Il est utilisé dans divers domaines, tels que le développement web, l'analyse de données, l'intelligence artificielle et l'automatisation. Python est apprécié pour sa facilité d'apprentissage et son large écosystème de bibliothèques.



Matplotlib est une bibliothèque de visualisation de données en Python qui permet de créer des graphiques statiques, animés et interactifs. Elle est extrêmement populaire pour la visualisation de données dans des domaines tels que l'analyse de données, les statistiques et l'apprentissage automatique. Matplotlib est largement utilisée pour tracer des graphiques, des diagrammes, des courbes, des histogrammes, des cartes thermiques, des diagrammes en nuages de points, etc.



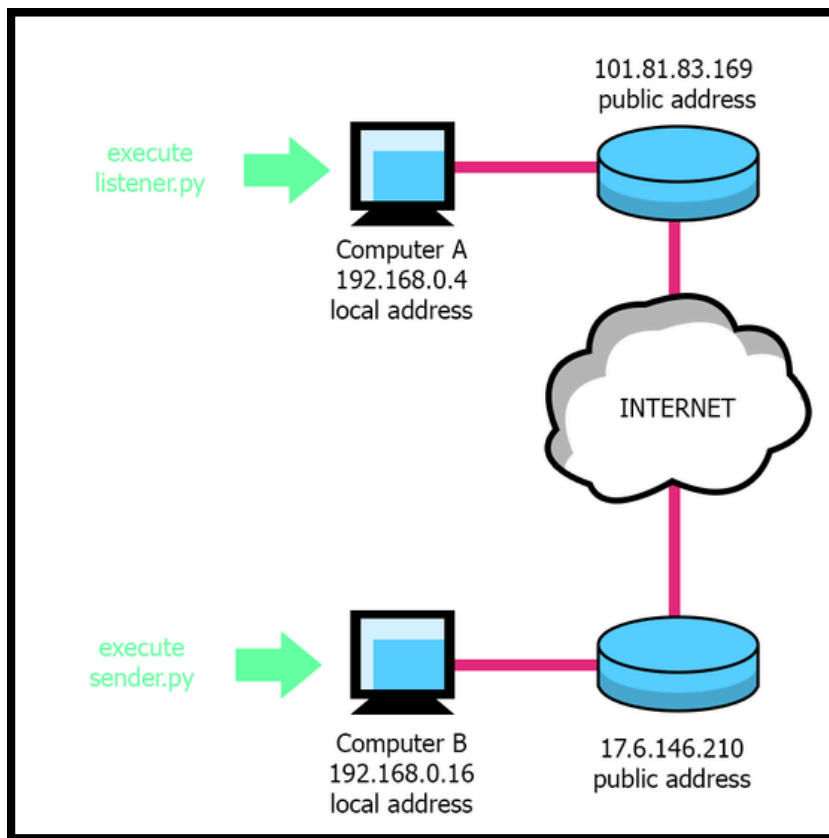
OpenCV (Open Source Computer Vision Library) :est une bibliothèque open-source largement utilisée pour les applications de traitement d'image et de vision par ordinateur. Elle propose une vaste gamme d'outils pour manipuler des images et des vidéos, y compris des fonctions pour la détection de visages, la reconnaissance d'objets, la manipulation de pixels, la transformation d'images, et bien plus encore. OpenCV est compatible avec plusieurs langages de programmation tels que Python, C++, et Java, et elle est couramment utilisée dans des domaines comme la robotique, la réalité augmentée, et la surveillance



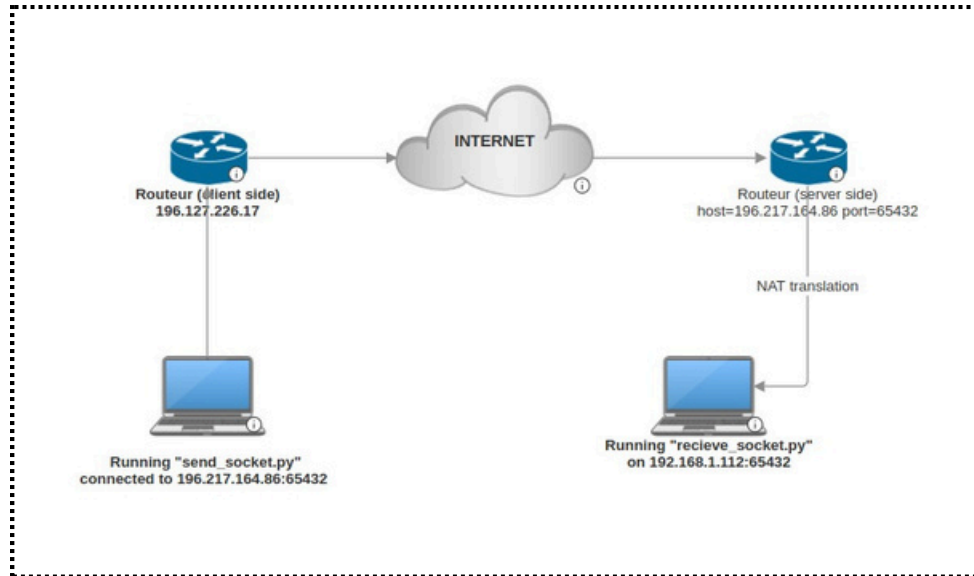
Azure Cloud est une plateforme de cloud computing développée par Microsoft, offrant une gamme de services cloud tels que le stockage, le calcul, l'analyse de données, l'intelligence artificielle et la gestion de bases de données. Azure permet aux entreprises de déployer, gérer et héberger des applications et services via des serveurs distants plutôt que d'utiliser des infrastructures locales. Il offre des solutions flexibles et évolutives adaptées à différents besoins, que ce soit pour des applications web, des services d'intelligence artificielle, ou des solutions de stockage.



sockets: sont des points de communication utilisés pour échanger des données entre des programmes, souvent sur un réseau informatique. Ils permettent de créer des connexions bidirectionnelles entre un serveur et un client, permettant l'envoi et la réception de données en temps réel. Les sockets peuvent être utilisés pour la communication locale sur un même ordinateur ou à travers un réseau, en utilisant des protocoles comme TCP/IP pour assurer la transmission fiable des informations.



Établissement de la Connexion Serveur-Client



Dans ce projet, nous avons mis en œuvre un système de communication réseau entre un serveur et un client en utilisant des sockets. L'objectif principal était de permettre une transmission fluide des données entre ces deux entités via un port réseau spécifique. Voici les étapes clés de notre démarche :

Exécution du code serveur (receiveSocket.py)

Sur le serveur, nous avons démarré un script Python nommé receiveSocket.py. Ce script est conçu pour écouter les connexions entrantes sur un port spécifique et gérer les données reçues. Ce rôle est central pour la réception et la gestion des requêtes envoyées par le client.

```
def start_server(host='192.168.1.112', port=65432):
    """Starts a TCP server to receive images."""
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as server_socket:
        server_socket.bind((host, port))
        server_socket.listen()
        print(f"Server listening on {host}:{port}...")

        conn, addr = server_socket.accept()
        with conn:
            print(f"Connected by {addr}")
```

```
if __name__ == "__main__":
    start_server()
```

Configuration du routeur

Nous avons configuré le routeur réseau pour effectuer un "port forwarding". Cette configuration permet de rediriger les requêtes provenant de l'extérieur vers notre serveur interne à travers un port réseau dédié. Nous avons également vérifié que ce port était ouvert et correctement exposé sur le serveur pour garantir la réception des connexions.

Advanced

Quick Start

Interface Setup

Advanced Setup

Access Management

Maintenance

Status

Help

Firewall

Routing

NAT

QoS

VLAN

ADSL

Virtual Server

Virtual Server Listing

Virtual Server for : Single IP Account

Rule Index : 1

Application : -

Protocol : ALL

Start Port Number : 65432

End Port Number : 65432

Local IP Address : 192.168.1.112

Rule	Application	Protocol	Start Port	End Port	Local IP Address
1	-	ALL	65432	65432	192.168.1.112
2	-	-	0	0	0.0.0.0
3	-	-	0	0	0.0.0.0
4	-	-	0	0	0.0.0.0
5	-	-	0	0	0.0.0.0
6	-	-	0	0	0.0.0.0
7	-	-	0	0	0.0.0.0
8	-	-	0	0	0.0.0.0
9	-	-	0	0	0.0.0.0
10	-	-	0	0	0.0.0.0
11	-	-	0	0	0.0.0.0
12	-	-	0	0	0.0.0.0

SAVE

DELETE

BACK

CANCEL

Exécution du code client (sendSocket.py)

Du côté du client, nous avons exécuté un script Python nommé sendSocket.py. Ce script envoie des données à l'adresse IP publique du routeur (ou l'adresse locale si en réseau privé), en spécifiant le même port que celui ouvert sur le serveur. Une fois la connexion établie, les données transmises sont gérées par le serveur via le code receiveSocket.py.

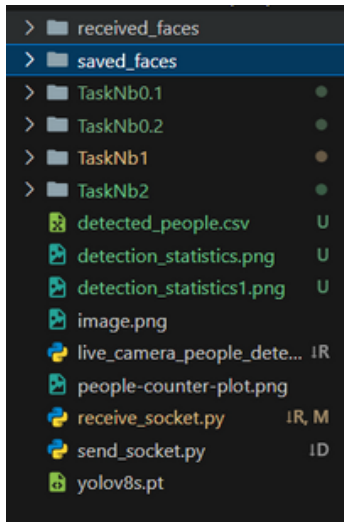
```
# Define the client class
class Client:
    def __init__(self, host='localhost', port=65432):
        self.host = host
        self.port = port
        self.cont = 0
        self.pos = {"x": [], "y": []}

    def is_clicked(self, x, y, button, pressed):
        if pressed:
            print('Clicked!')
            self.pos["x"].append(x)
            self.pos["y"].append(y)
            self.cont += 1
            if self.cont == 2:
                return False # stop the listener after two clicks

    def start(self):
        # Start the client socket
        client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        client_socket.connect((self.host, self.port))
        print("Connected to the server.")
```

```
# Start the client
if __name__ == "__main__":
    client = Client()
    client.start()
```

Acquisition et Prétraitement des Images



Ce projet utilise un modèle YOLOv8 pour détecter des personnes en temps réel, avec des dossiers pour stocker les visages détectés et diverses tâches. Les scripts Python gèrent la détection en direct et l'échange de données via des sockets. Des fichiers d'image et CSV permettent de visualiser et analyser les statistiques de détection.

Pour le TaskNB0.1:

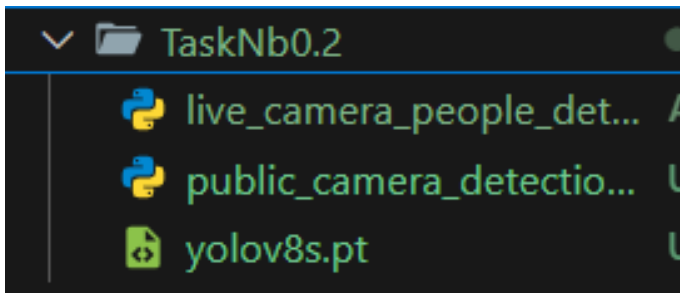
```
import cv2
import matplotlib.pyplot as plt
import pandas as pd
from ultralytics import YOLO
import mss
import pynput
import time
```

cv2: utilisé pour charger et traiter les images.
 matplotlib.pyplot: utilisé pour afficher les images avec les détections.
 pandas: utilisé pour organiser et afficher les données de détection.
 YOLO de ultralytics: bibliothèque qui fournit le modèle YOLOv8 pour la détection d'objets.

Ce code est utilisé pour :

- **Détecter des personnes dans une image** : le modèle YOLOv8 identifie et marque les personnes dans l'image.
- **Visualiser les détections** : l'image est affichée avec des rectangles autour des personnes détectées.
- **Enregistrer les informations de détection** : les données (classe, confiance, coordonnées) sont organisées dans un DataFrame, ce qui permet une analyse ou un traitement ultérieur.

Pour le TaskNB0.2:



Ce code contient deux scripts principaux pour la détection de personnes en utilisant des flux vidéo, que ce soit depuis une webcam locale ou une caméra publique en ligne.

Voici une description de chaque partie :

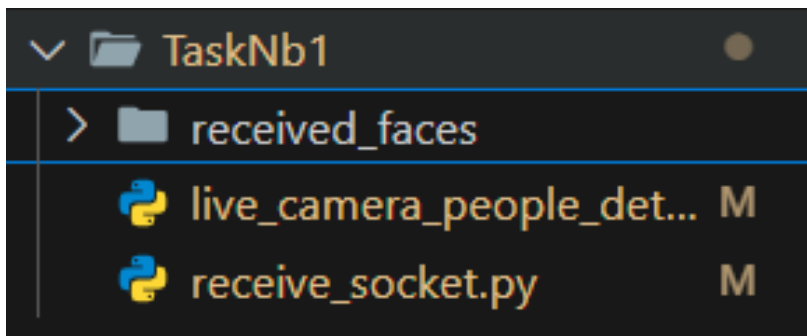
1. **detect_people_live** : Détection de personnes en direct depuis la webcam

- Utilise OpenCV pour accéder à la webcam par défaut et lire les images en temps réel.
- Charge le modèle YOLOv8 (fourni par ultralytics) pour détecter des objets dans chaque image de la webcam.
- Pour chaque image, le modèle détecte les objets et conserve uniquement ceux qui correspondent à la classe "personne".
- Dessine des rectangles autour des personnes détectées, accompagnés de la confiance de détection, puis affiche l'image traitée.
- Le script se termine si l'utilisateur appuie sur la touche q.

2. **Classe Client** : Détection de personnes depuis une caméra publique

- Utilise OpenCV pour se connecter à une caméra en ligne, spécifiée par une URL (camera_url).
- Initialise un détecteur de personnes basé sur le descripteur HOG (Histogram of Oriented Gradients), ce qui est moins précis que YOLO mais fonctionne sans modèle pré-entraîné.
- Récupère les images de la caméra publique, redimensionne l'image pour une détection plus rapide, puis utilise le détecteur HOG pour détecter des personnes.
- Dessine des rectangles autour des personnes détectées et affiche l'image en direct avec les détections.
- Affiche le nombre de personnes détectées pour chaque image capturée.
- Se termine lorsque l'utilisateur appuie sur la touche q.

Pour le TaskNB1:



Ce code est conçu pour la surveillance en temps réel et pour transmettre des images de visages détectées à un serveur. Cela peut être utile dans des applications de sécurité pour capturer et archiver des images de visages de personnes détectées en temps réel. Le serveur enregistre chaque visage reçu dans un dossier spécifique pour une analyse ou un traitement ultérieur.

Le code contient deux scripts principaux:

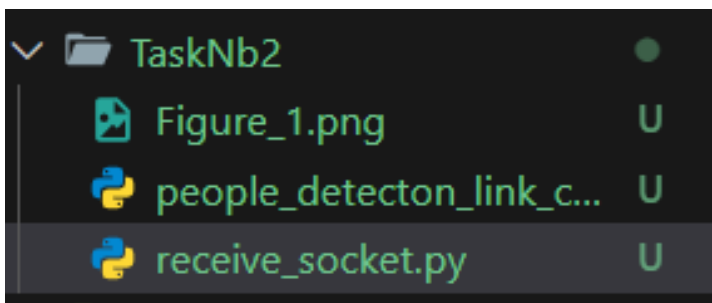
1. **detect_people_live :**

- Détection de personnes en direct et envoi d'images de visages au serveur
- Modèle YOLOv8 : Utilisé pour détecter les objets dans les images de la webcam. Ce modèle est pré-entraîné et peut identifier plusieurs classes, mais ici, on se concentre uniquement sur la classe "personne".
- Détection et suivi de personnes: Pour chaque nouvelle personne détectée, le code vérifie si elle a été détectée auparavant (en comparant les coordonnées de la boîte englobante avec une tolérance de 90 pixels).
- Si une personne est détectée pour la première fois, elle est enregistrée avec un horodatage.
- Si la personne reste détectée pendant 3 secondes, une image de son visage est sauvegardée localement et envoyée au serveur.
- Envoi d'image au serveur : L'image du visage sauvegardée est lue et envoyée au serveur via une connexion socket. La taille de l'image est d'abord envoyée pour que le serveur puisse préparer la réception.
- Interface : La détection en temps réel est affichée avec des rectangles verts pour les personnes déjà détectées et sauvegardées, et des rectangles rouges pour les nouvelles personnes en cours de détection.

2. start_server :

- Réception et sauvegarde des images de visages envoyées par le client
- Serveur TCP : Écoute les connexions sur une adresse IP et un port spécifiés pour recevoir des images de visages envoyées par le client.
- Réception de la taille de l'image : Le serveur reçoit d'abord la taille de l'image en 8 octets pour préparer la réception de la donnée complète.
- Réception de l'image : Le serveur accumule les paquets de données jusqu'à obtenir la taille d'image complète, puis il convertit le flux de bytes en image.
- Sauvegarde : Une fois l'image décodée, elle est sauvegardée dans le dossier received_faces avec un identifiant unique.
- Sécurité : Le serveur valide que l'image ne dépasse pas 10 Mo pour éviter des envois non contrôlés.

Pour le TaskNB2:



Le code est conçu pour cree un serveur qui reçoit et enregistre les images contenant des personnes détectées par une application cliente. Voici une description détaillée des fonctionnalités :

1-Côté client :

- Utilise un modèle YOLO (éventuellement ultralytique) pour détecter des personnes dans un flux vidéo en direct (ou une source d'image).
- Capture des images du flux vidéo.
- Identifie les cadres de délimitation autour des personnes détectées dans chaque image.
- Pour chaque personne détectée :
 - Vérifie si la personne a été vue récemment (dans un seuil de tolérance).
 - S'il s'agit d'une nouvelle personne ou d'une personne non vue récemment (3 secondes par défaut) :
 - Enregistre l'image du visage dans un fichier séparé.
 - Envoie l'image du visage enregistrée au serveur à l'aide d'une connexion socket.

2-Côté serveur :

- Écoute les connexions entrantes sur un port spécifié (65432 par défaut).
- Lorsqu'un client se connecte, le serveur reçoit des données en deux parties :
 - Tout d'abord, la taille des données de l'image (en octets) est reçue.
 - Ensuite, les données d'image réelles sont reçues.
- Le serveur valide la taille de l'image pour éviter tout transfert de données volumineux ou non valide.
- Si l'image est valide, le serveur l'enregistre avec un nom de fichier unique dans un répertoire nommé « received_faces ».
- Le serveur enregistre l'événement de détection avec un horodatage dans un fichier CSV nommé « detected_people.csv ».
- Le serveur garde une trace des détections par minute pour un traçage en direct.
- Un graphique en direct facultatif est tracé pour visualiser le nombre de personnes détectées par minute.
- À la fin du programme, le serveur enregistre le graphique des statistiques de détection sous forme d'image (« detection_statistics.png »).

 `live_camera_people_dete... IR`

Ce code est conçu pour des applications de **surveillance intelligente** en temps réel. Il utilise la détection de personnes avec le modèle YOLO via une webcam et permet de capturer, sauvegarder et envoyer automatiquement les visages détectés à un serveur. Ce type de solution est particulièrement utile dans des contextes de **sécurité** ou de **contrôle d'accès**, où il est crucial de surveiller les personnes entrant dans une zone sensible. Grâce à ce système, les visages sont identifiés et sauvegardés avec une gestion de la redondance, ce qui évite les captures multiples d'une même personne en peu de temps. L'envoi des images au serveur permet d'analyser et de vérifier les identités en temps réel, contribuant ainsi à améliorer la sécurité et la gestion des accès de manière efficace.

 `receive_socket.py IR, M`

Ce code est conçu pour la **surveillance en temps réel** et **l'identification automatique de personnes** à partir d'un flux de caméra en direct. Utilisant un modèle YOLO (You Only Look Once) pour la détection d'objets, il permet de repérer les personnes dans une scène, de capturer leurs visages, et d'envoyer ces images à un serveur pour analyse ou stockage. Une fois une personne détectée, le système vérifie si elle a déjà été enregistrée en comparant les coordonnées de détection. Si la personne est nouvelle ou si elle n'a pas encore été sauvegardée, son visage est capturé et envoyé au serveur.

Ce code est particulièrement utile pour des applications de **sécurité**, telles que le suivi en temps réel dans les espaces publics, le contrôle d'accès dans les entreprises, ou encore l'authentification basée sur l'apparence dans des zones sensibles. L'utilisation d'un délai de 3 secondes avant d'enregistrer un visage permet d'éviter les doublons. Ce système améliore l'efficacité des processus de sécurité en réduisant les interventions humaines, en automatisant la détection et en facilitant le suivi des mouvements des personnes en temps réel.



send_socket.py

ID

Ce code implémente un client de **surveillance d'écran en temps réel** qui capture des images d'une zone spécifique de l'écran définie par l'utilisateur, envoie ces images à un serveur, et reçoit un retour d'information. La sélection de la zone d'écran se fait en cliquant deux fois aux coins opposés de la zone à capturer. Une fois la zone définie, le client capture cette portion de l'écran périodiquement (toutes les 5 secondes), convertit l'image en tableau de données et l'envoie au serveur via une connexion socket.

Ce programme est utile pour diverses applications comme :

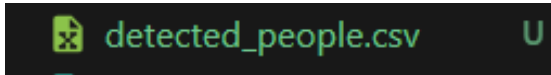
- - **Surveillance en temps réel** : Permet de surveiller l'activité sur un écran distant.
- - **Analyse de contenu** : Le serveur peut analyser les captures d'écran, par exemple pour détecter le nombre de personnes ou tout autre élément sur l'écran.
- - **Télémaintenance et support technique** : Capturer et analyser l'activité de l'écran de l'utilisateur pour offrir une assistance technique ou pour surveiller des opérations.

Cette approche réduit les besoins d'intervention humaine dans la surveillance continue, optimise les processus de détection à distance et aide à obtenir des informations en temps réel pour la sécurité, l'assistance ou l'analyse de contenu.

Détection et Traitement des Données

La détection et le traitement des données sont des étapes cruciales dans ce projet. Voici un aperçu des principales étapes et techniques impliquées :

1.Detected_people.csv :



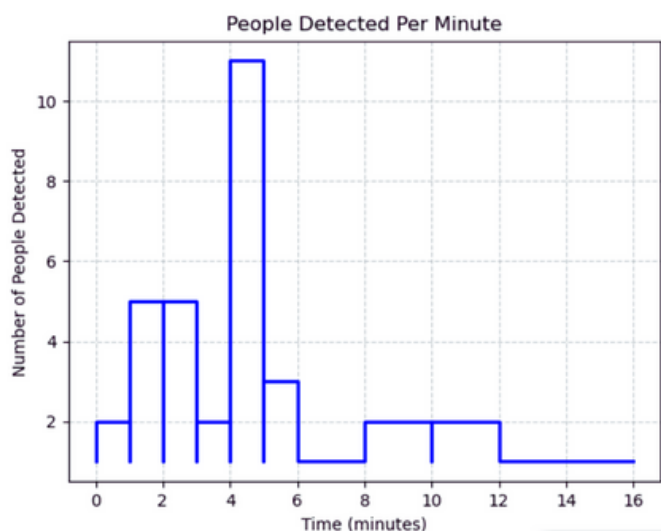
Fichier CSV contenant probablement des données sur les personnes détectées dans le cadre du projet de détection.

Ce fichier pourrait contenir des informations telles que l'ID des personnes, les coordonnées de détection, les horodatages, ou d'autres métadonnées.

1	Face_ID, Timestamp
2	45, 2024-11-08 22:05:11
3	46, 2024-11-08 22:05:13
4	47, 2024-11-08 22:05:15
5	48, 2024-11-08 22:05:30
6	49, 2024-11-08 22:05:37
7	50, 2024-11-08 22:05:39
8	51, 2024-11-08 22:05:41
9	52, 2024-11-08 22:05:43
10	53, 2024-11-08 22:05:45
11	54, 2024-11-08 22:05:48
12	55, 2024-11-08 22:05:55
13	

2. Figure 1.png :

Fichier image au format PNG.



Affichez un graphique, consultez les résultats de la détection ou un exemple d'image avec les personnes détectées.

3.people detecton link camera:

La partie du code qui effectue la détection des personnes à partir du flux de la caméra commence par ouvrir le flux vidéo en utilisant `cv2.VideoCapture`, avec un lien de caméra publique. Ensuite, dans une boucle, chaque image est capturée et convertie en format RGB pour être compatible avec le modèle YOLO. Le modèle YOLO, initialisé au début du script, effectue ensuite la détection des objets dans l'image, retournant les coordonnées de chaque "boîte englobante" autour des objets détectés. Seuls les objets classés comme "personne" sont retenus pour la suite du traitement. Pour chaque personne détectée, le code vérifie si elle a déjà été détectée récemment en comparant les positions des boîtes englobantes. Si la personne est nouvelle ou n'a pas été sauvegardée dans les trois dernières secondes, son visage est capturé, sauvegardé, et envoyé au serveur. Le flux vidéo est ensuite affiché avec des rectangles rouges pour les nouvelles personnes et verts pour celles déjà sauvegardées, jusqu'à ce que l'utilisateur appuie sur la touche "q" pour arrêter.

```
def detect_people_live():
    # Use the public stream URL instead of the webcam
    cap = cv2.VideoCapture("https://csea-me-webcam.cse.umn.edu/mjpg/video.mjpg") # Replace with your public stream link

    if not cap.isOpened():
        print("Error: Could not open video stream.")
        return

    person_id_counter = 1 # To assign a unique ID for each new person

    # Connect to the server
    client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    client_socket.connect(('127.0.0.1', 65432)) # Adjust host and port as needed

    try:
        while True:
            # Capture frame-by-frame from the stream
            ret, frame = cap.read()
            if not ret:
                print("Error: Failed to capture image from video stream.")
                break

            # Convert BGR to RGB for the YOLO model "YOLO": Unknown word.
            img_rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)

            # Perform detection using the YOLO model "YOLO": Unknown word.
            results = model(img_rgb)

            # Process detection results
            for result in results:
                boxes = result.boxes
                for box in boxes:
```

4.receive_socket.py:

Ce code met en place un serveur TCP qui reçoit des images de visages détectés, les enregistre, et suit les statistiques de détection. Le serveur écoute sur l'adresse locale ('127.0.0.1') et le port '65432'. Lorsqu'une connexion est établie, il reçoit d'abord la taille de l'image envoyée en octets (8 octets). Ensuite, il récupère l'image elle-même en paquets de données jusqu'à atteindre la taille spécifiée. L'image reçue est alors décodée et sauvegardée dans un dossier nommé 'received_faces' avec un identifiant unique. Parallèlement, une entrée est ajoutée à un fichier CSV ('detected_people.csv') pour enregistrer l'identifiant du visage et l'horodatage de la détection. Le code garde également une trace des détections par minute, ce qui permet de créer un graphique en temps réel qui montre le nombre de personnes détectées par minute sous forme de courbe à pas, affichée dans une fenêtre interactive. Ce graphique est mis à jour en continu, puis sauvegardé en tant qu'image ('detection_statistics1.png') à la fin de l'exécution du serveur.

```
def start_server(host='127.0.0.1', port=65432):
    """Starts a TCP server to receive images and log detections."""
    # Initialize detection statistics
    detections_per_minute = {}
    start_time = time.time()

    # Prepare the CSV file to log detections
    csv_file = "detected_people.csv"
    with open(csv_file, mode='w', newline='') as file:
        writer = csv.writer(file)
        writer.writerow(["Face_ID", "Timestamp"]) # Write CSV headers

    # Prepare for live plotting with a stepped line style
    plt.ion()
    fig, ax = plt.subplots()
    ax.set_xlabel("Time (minutes)")      "xlabel": Unknown word.
    ax.set_ylabel("Number of People Detected")  "ylabel": Unknown word.
    ax.set_title(["People Detected Per Minute"])
    x_data, y_data = [], []

    def update_chart():
        """Updates the chart with current detection statistics."""
        current_time = time.time()
        elapsed_minutes = int((current_time - start_time) / 60)
```

Intégration Cloud -Azure-

Azure, développé par Microsoft, est une plateforme de cloud computing leader sur le marché, offrant une large gamme de services pour créer, déployer et gérer des applications et des infrastructures à travers des centres de données gérés par Microsoft dans le monde entier. Lancée en 2010, Azure est devenue l'une des plateformes cloud les plus populaires, aux côtés d'AWS et de Google Cloud, grâce à sa scalabilité, sa sécurité, et sa flexibilité.

Qu'est-ce qu'Azure Cloud ?

Azure est une plateforme de cloud public qui permet aux entreprises et aux développeurs d'accéder à des ressources informatiques, de stockage, de mise en réseau, et à des capacités avancées comme l'intelligence artificielle, à la demande. Elle prend en charge une variété de systèmes d'exploitation, de langages de programmation, et de frameworks, ce qui la rend adaptable aux besoins de tout type d'organisation.

Azure offre des services répartis sur trois principaux modèles de cloud computing :

1. **Infrastructure as a Service (IaaS)** : Fournit des ressources informatiques virtualisées via Internet.
2. **Platform as a Service (PaaS)** : Permet aux développeurs de créer, tester et déployer des applications sans gérer l'infrastructure sous-jacente.
3. **Software as a Service (SaaS)** : Propose des applications logicielles accessibles via Internet sur une base d'abonnement.

Caractéristiques principales d'Azure

1. Couverture mondiale :

- Azure est présent dans 60+ régions et dispose de 200+ centres de données à travers le monde, permettant un déploiement des ressources proche des utilisateurs finaux.

2. Scalabilité et flexibilité :

- Azure prend en charge l'auto-scalabilité et des modèles de tarification flexibles, permettant d'ajuster les ressources en fonction des besoins.

3. Écosystème intégré :

- S'intègre parfaitement aux outils Microsoft tels que **Office 365**, **Active Directory**, et **Visual Studio**.
- Prend également en charge des technologies open-source et tierces.



Services populaires d’Azure

- **Services de calcul :**

Azure Virtual Machines (VM), Kubernetes (AKS) et App Services pour l’hébergement d’applications.

- **Solutions de stockage :**

Blob Storage, Azure Files, et Data Lake pour répondre aux besoins de stockage.

- **Services de bases de données :**

Azure SQL Database, Cosmos DB (NoSQL), et Azure Database pour PostgreSQL/MySQL.

- **Mise en réseau :**

Réseaux Virtuels (VNet), Load Balancers, et Content Delivery Network (CDN) pour la connectivité.

- **Analyse et Big Data :**

Azure Synapse Analytics, HDInsight, et Databricks pour le traitement des données volumineuses.

- **IA et IoT :**

Azure AI et IoT Hub pour des applications connectées et intelligentes.

Création et Configuration de la Machine Virtuelle

- **Création de la VM :**

- Description des étapes suivies pour créer une machine virtuelle dans Azure.

- Paramètres sélectionnés pour la VM :

- Type : Standard B1s.
- Système d'exploitation : (exemple) Ubuntu Server 22.04.

Microsoft Azure

Home > Free services >

Create a virtual machine

Basics Tags Review > create

Create a virtual machine that runs Linux or Windows. Select an image from Azure marketplace or use your own customized image. Complete the Basics tab then Review > create to provision a virtual machine with default parameters or review each tab for full customization. [Learn more](#)

Project details

Select the subscription to manage deployed resources and costs. Use resource groups like folders to organize and manage all your resources.

Subscription * Azure subscription 1

Resource group * linux

[Create new](#)

Instance details

Virtual machine name * Ubuntu-projet-SE

Region * (Europe) North Europe

Image * Ubuntu Server 22.04 LTS - x64 Gen2

Size * Standard_B1s - 1 vcpu, 1 GB memory (\$8.25/month) (free services eligible)

Administrator account

Authentication type * SSH public key

☒ SSH public key

☐ Password

Azure now automatically generates an SSH key pair for you and allows you to store it for future use. It is a fast, simple, and secure way to connect to your

< Previous Next > Tags Review > create

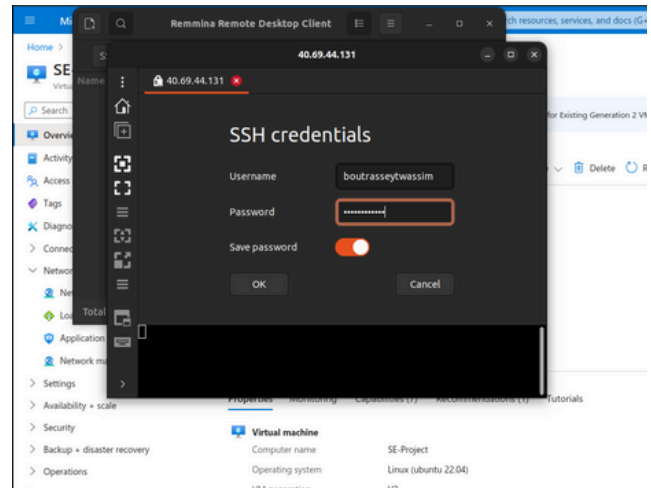
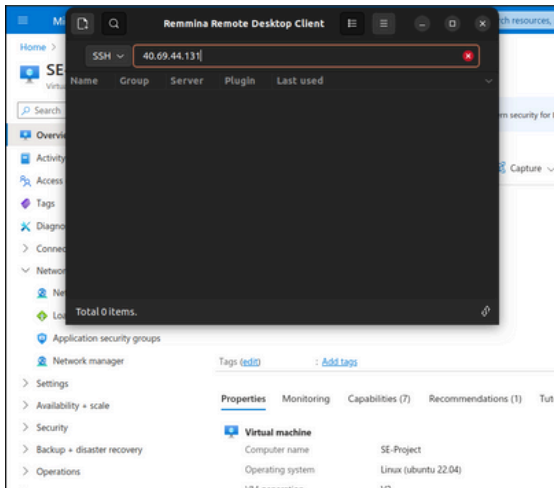
[Give feedback](#)

Une fois la machine virtuelle est crée on passe alors à la configuration du réseau de cette dernière , ainsi de l’installation des dépendences .Après avoir choisit le mode de connexion et les ressources dont on a besoin .

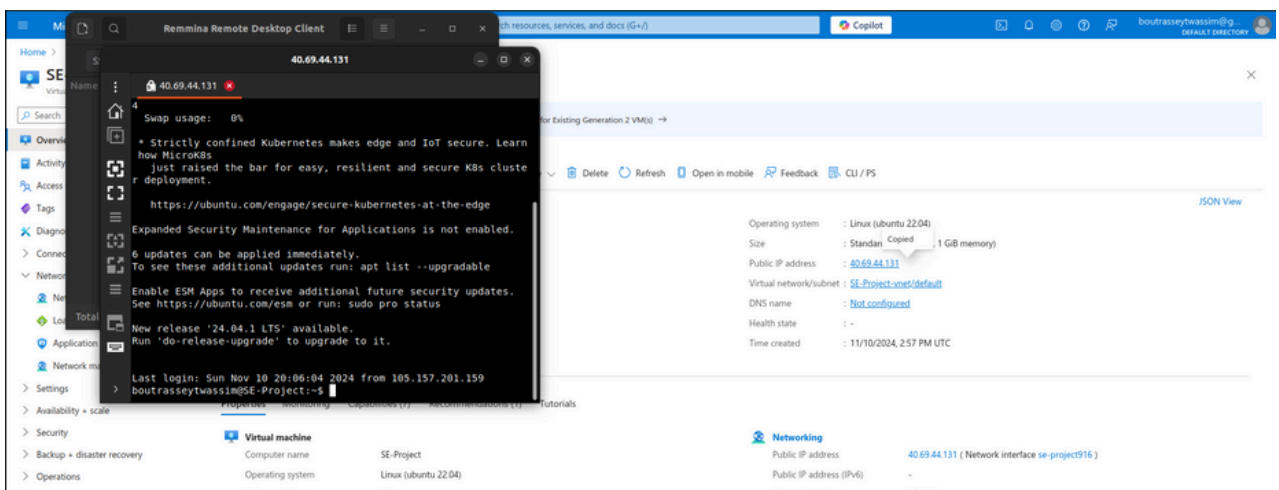
Configurer l’authentification et les utilisateurs

Pour établir la connexion avec ma machine virtuelle dans azure , on a utilisé l’outil **Remmina** qui permet d’accéder à la vm à travers une machine linux distante .





On entre l'adresse ip de notre machine pour faire une liaison ssh .Une fois liée on nous demande de entrer le **Username** et **Password** pour s'authentifier et avoir l'accès à la machine

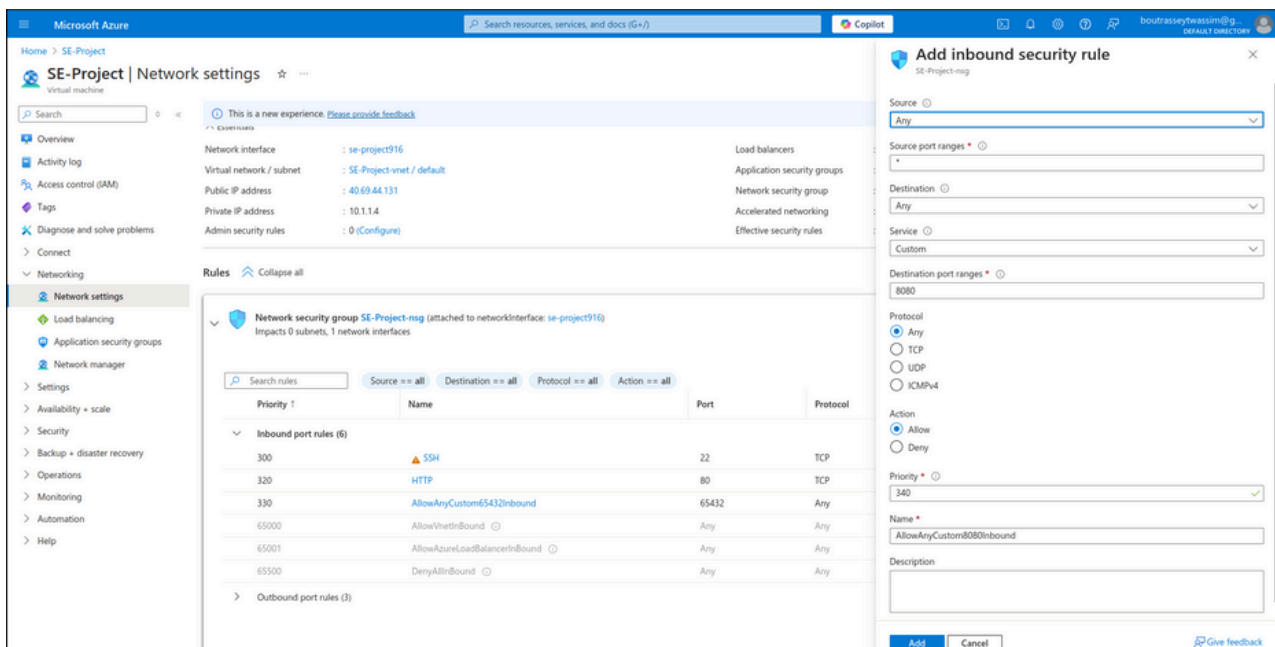
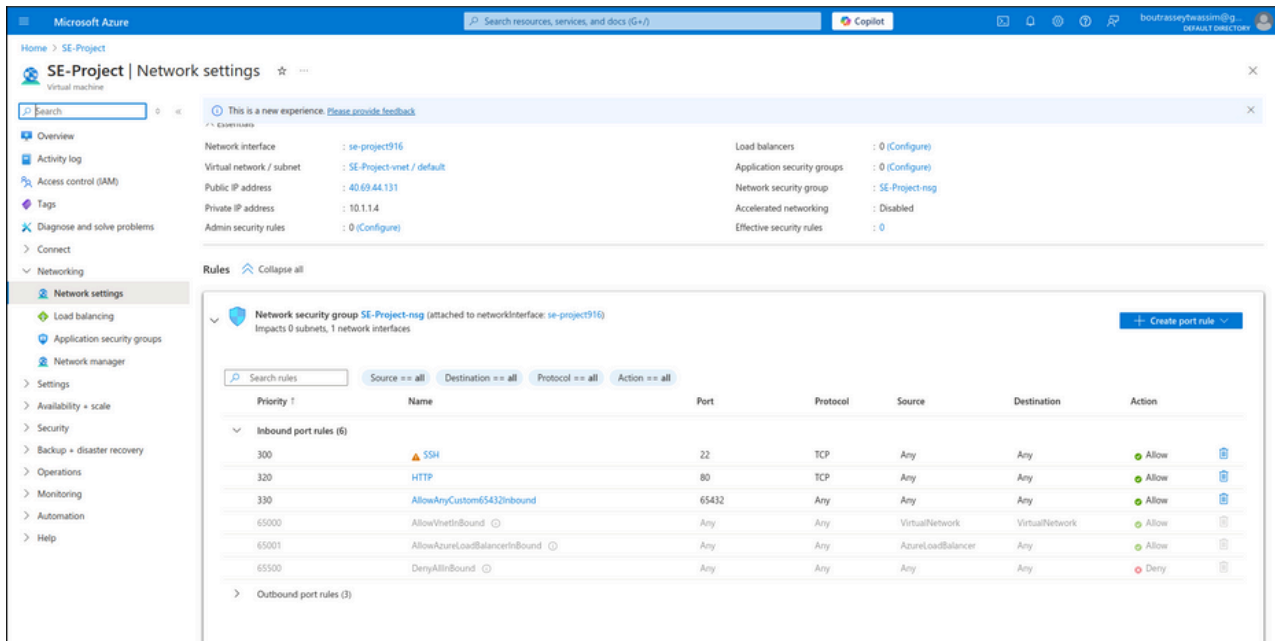


Configurer le réseau

On a modifié les paramètres du Groupe de Sécurité Réseau (NSG) pour permettre l'accès à certains services via des ports spécifiques, comme par exemple le port : 65432 pour faire une liaison **Socket** avec les clients

Les règles de sécurité réseau configurées pour la machine virtuelle contrôlent le trafic entrant en fonction des ports autorisés. Les principales règles configurées sont :

- **Port 22 (SSH)** : Permet les connexions sécurisées via SSH pour l'administration à distance.
- **Port 80 (HTTP)** : Permet l'accès web pour des applications ou services HTTP.
- **Port 65432** : Port personnalisé ajouté pour la réception des requêtes arrivées dans la session de **Socket**
- Les autres règles par défaut pour Azure Load Balancer et autres communications internes sont également visibles.

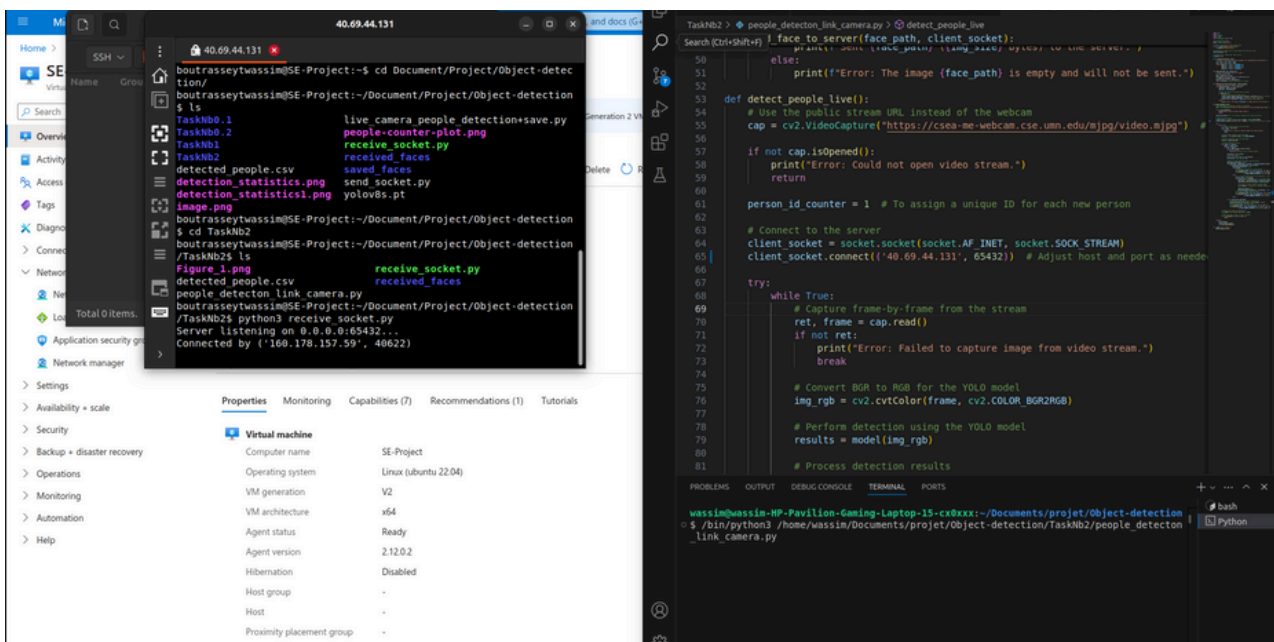
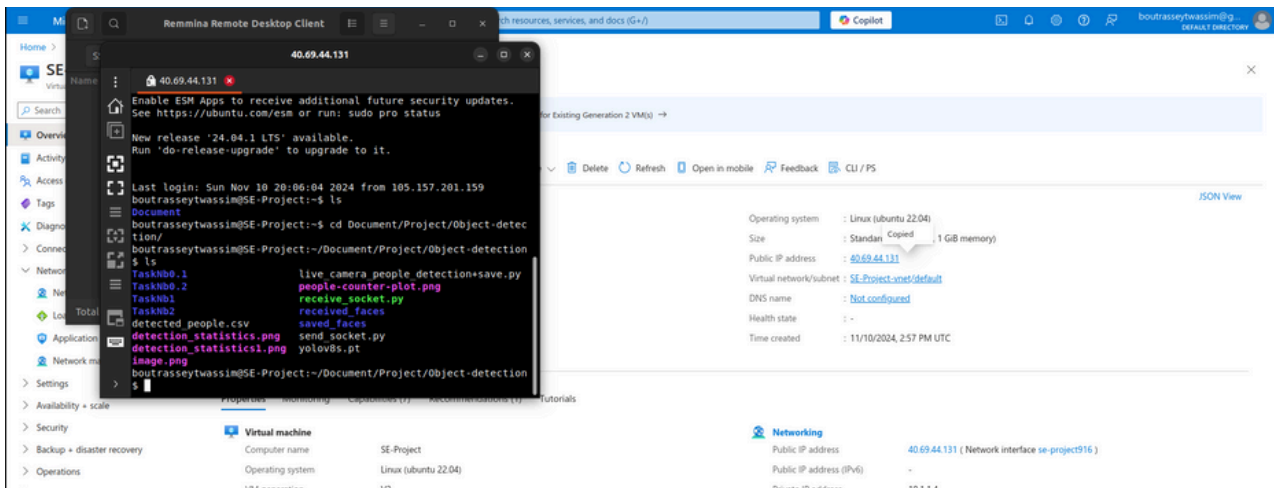


Installation des dépendances et clonage du projet avec Git

Ces images montrent les étapes nécessaires pour préparer l'environnement en installant les dépendances requises pour le projet. Les actions réalisées incluent :

- Mise à jour de la liste des paquets avec la commande `sudo apt update`.
- Installation des outils essentiels comme **pip** (gestionnaire de paquets **Python**) et **git** (gestion des versions).
- Installation des bibliothèques spécifiques au projet à l'aide de **pip**, notamment :
 - **matplotlib** : pour les visualisations graphiques.
 - **ultralitics** : pour utiliser le modèle **YOLO**.
 - **pandas** : pour la manipulation et l'analyse des données.
 - **opencv-python** : pour le traitement des images.

Cette configuration garantit que toutes les dépendances nécessaires sont disponibles pour exécuter le projet.

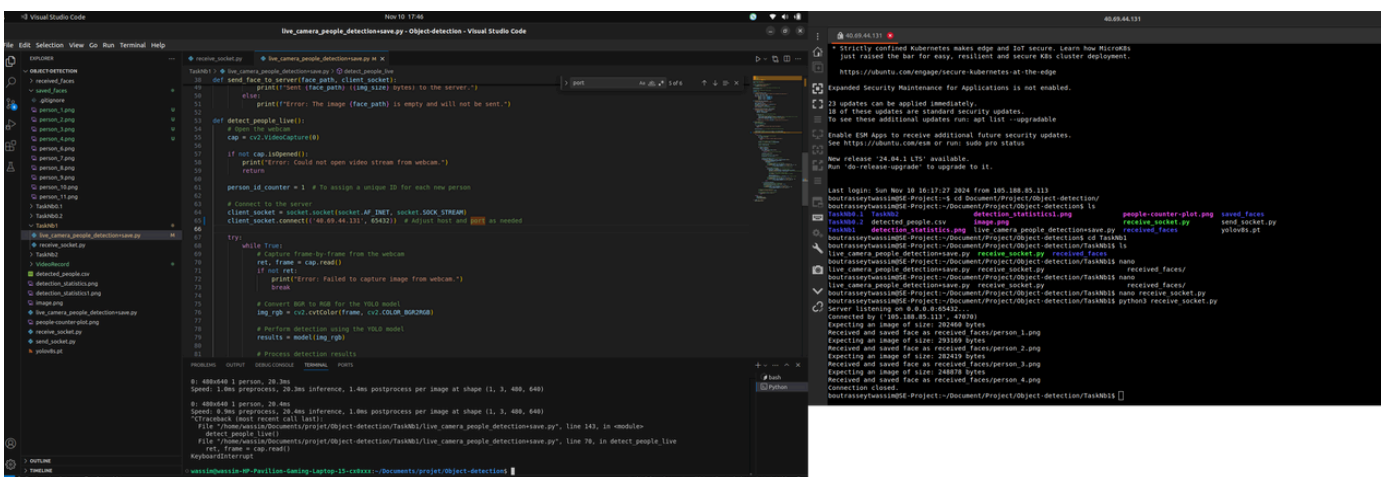


l'utilisation de **Git** pour télécharger les fichiers du projet depuis un dépôt distant. Les étapes incluent :

- Utilisation de la commande **git clone** <<https://github.com/San-AMRANI/Object-detection.git>> pour récupérer le code source.
- Navigation vers le répertoire du projet à l'aide de la commande **cd** <**Object-detection**>.

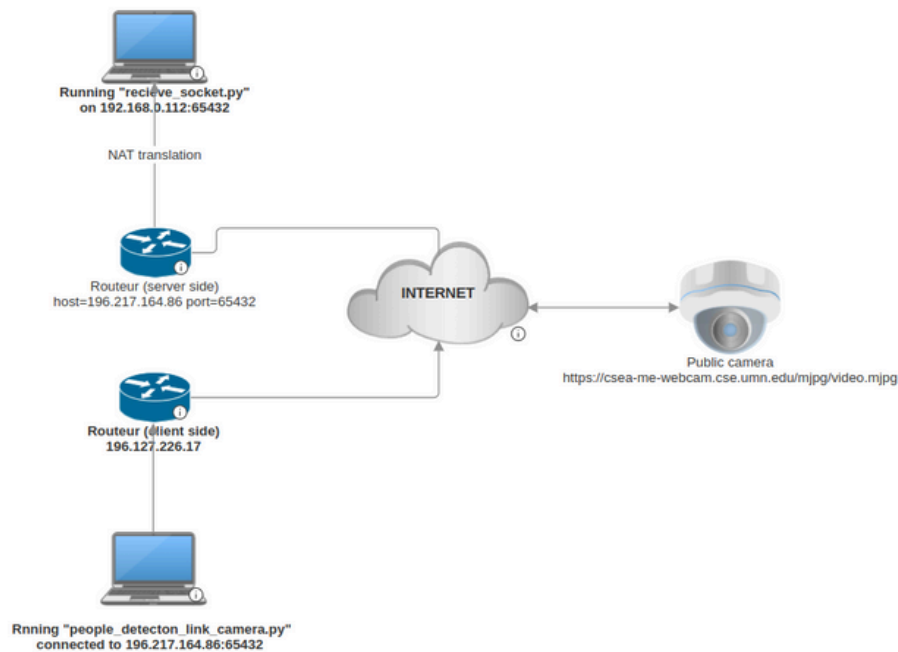
Ces étapes permettent de rendre le projet localement accessible pour l'exécution et les tests.

Et finalement l'exécution du code avec la commande **python3 receive_socket.py** pour lancer le socket server en attendant les socket clients de se connecter . La dernière figure illustre une démonstration



Application

Après la mise en place de la topology Réseau et interconnection Client-Serveur via l'internet



Les fichier requis;

- **people_detected_link_camera.py** : Ce script détecte les personnes dans une vidéo en streaming, capture leur visage, et envoie l'image au serveur.
- **receive_socket.py** : Ce script reçoit les images des visages envoyées par le premier script, les enregistre, et met à jour les statistiques des détections.

Étape n°1: Lancement du Serveur de Réception

Avant d'exécuter le script client, le serveur doit être démarré en utilisant receive_socket.py.

- Adresse du serveur : 192.168.1.112
- Port d'écoute : 65432

Le script écoute les connexions entrantes sur le réseau local qui sera deriger depuis le réseau wan par le routeur.

```
sanamrani@SanUnix:~/Desktop/Unix Proj/Peaple Detection/TaskNb2$ python3 receive_socket.py
Server listening on 192.168.1.112:65432...
```

```

sanamrani@SanUnix:~/Desktop/Unix Proj/Peaple Detection/TaskNb2$ python3 receive_socket.py
Server listening on 192.168.1.112:65432...
Connected by ('192.168.1.112', 53228)
Expecting an image of size: 10528 bytes
Received and saved face as received_faces/person_1.png
Expecting an image of size: 3910 bytes
Received and saved face as received_faces/person_2.png
Expecting an image of size: 6095 bytes

```

Étape n°2: Configuration de la Connexion Internet

Pour permettre une connexion depuis un client situé sur un autre réseau :

1. Configurez le router du serveur pour activer le port forwarding.
 - Redirigez le port 65432 vers 192.168.1.112:65432 (IP locale du serveur).
2. Le client utilise l'adresse IP publique du réseau du serveur : 196.217.164.86.

Étape n°3: Exécution du Script Client

Le script `people_detected_link_camera.py` est configuré pour se connecter au serveur via son IP publique.

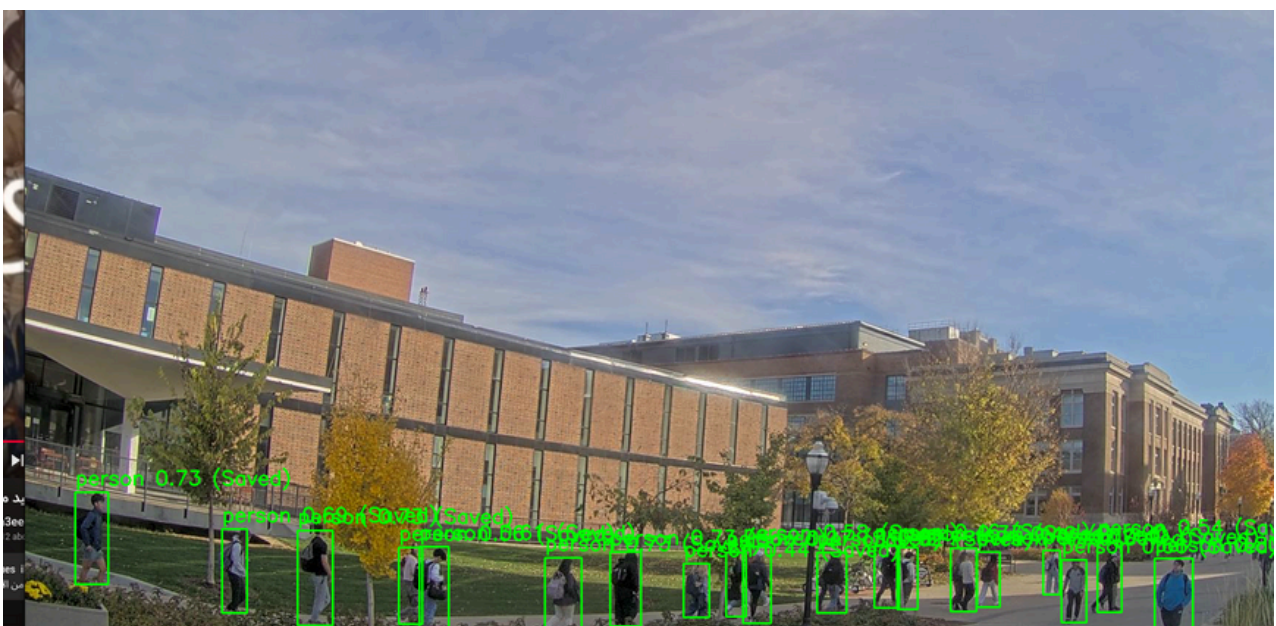
- Adresse cible : 196.217.164.86
- Port cible : 65432

Le script tentera de se connecter au serveur. Une fois connecté, il commencera à transmettre les images capturées.

Étape n°4: Détection en Temps Réel

Le script client analyse chaque cadre vidéo pour détecter les personnes.

- Le modèle YOLOv8 détecte les objets dans chaque image du flux vidéo.
- Les visages des personnes détectées sont capturés et sauvegardés temporairement.
- Les visages sont transmis au serveur si :
 - La personne est détectée pour la première fois.
 - Un délai suffisant s'est écoulé depuis la dernière détection de cette personne.



L'image montre un cadre où les personnes détectées sont entourées de rectangles (vert pour les personnes déjà envoyés, rouge pour les nouvelles personnes).

Étape n°5: Réception des Images sur le Serveur

Le script `receive_socket.py` reçoit les images envoyées par le client :

- Chaque image est décodée et sauvegardée dans le dossier `received_faces`.
- Une entrée est ajoutée dans `detected_people.csv` avec :
 - L'ID de l'image.
 - Un horodatage précis.
- Les statistiques des détections sont mises à jour et affichées sous forme de graphique en temps réel.

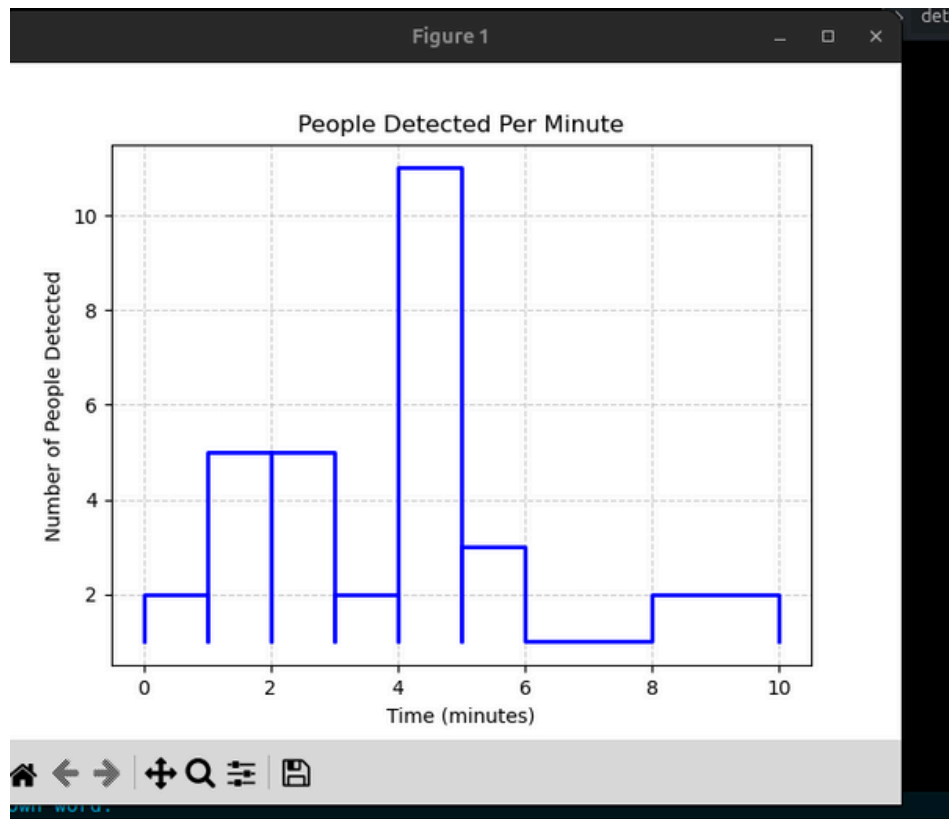
Exemple d'image reçue :



```
sanamrani@SanUnix:~/Desktop/Unix Proj/Peaple Detection/  
Server listening on 192.168.1.112:65432...  
Connected by ('192.168.1.112', 53228)  
Expecting an image of size: 10528 bytes  
Received and saved face as received_faces/person_1.png  
Expecting an image of size: 3910 bytes  
Received and saved face as received_faces/person_2.png  
Expecting an image of size: 6095 bytes
```

Étape n°6: 6. Mise à Jour des Statistiques

Le serveur génère un graphique en temps réel qui affiche le nombre de visages détectés par minute. Exemple de graphique généré :



Résultats et Analyse

- Précision des Détections : Le modèle YOLOv8 a détecté efficacement les personnes dans les flux vidéo.
- Connectivité Internet : La transmission des images fonctionne correctement via une connexion internet grâce au port forwarding.
- Gestion des Données : Les images et statistiques sont enregistrées avec un taux de succès élevé.

Conclusion

Le système développé démontre une application pratique et efficace de la détection de personnes en temps réel combinée à la transmission d'images via Internet. Cette technologie peut être déployée dans divers domaines tels que :

- **Surveillance de Sécurité** : Utilisé dans des zones sensibles (banques, aéroports, entrepôts) pour identifier et suivre les individus en temps réel.
- **Systèmes de Contrôle d'Accès** : Intégré avec des bases de données pour vérifier l'identité des personnes détectées.
- **Analyse de Flux de Clients** : Dans les commerces et centres commerciaux pour analyser les comportements des clients et optimiser les opérations.

Pistes de Développement

Pour étendre les fonctionnalités et renforcer les performances, plusieurs améliorations peuvent être envisagées :

1. **Cryptage des Données** : Ajouter une couche de sécurité en utilisant des protocoles comme TLS pour sécuriser les communications entre le client et le serveur.
2. **Reconnaissance Faciale** : Coupler le système de détection avec une fonctionnalité de reconnaissance faciale pour identifier précisément les personnes.
3. **Gestion Multicaméra** : Supporter plusieurs flux vidéo en simultané pour couvrir une plus grande zone de surveillance.
4. **Notifications en Temps Réel** : Intégrer des alertes (par e-mail ou SMS) pour notifier les utilisateurs en cas de détections spécifiques.
5. **Interface Web** : Développer une interface utilisateur permettant de visualiser les détections en direct, de consulter l'historique des détections, et de configurer le système.
6. **Analyse Avancée** : Ajouter des outils d'analyse comme le comptage de personnes, le suivi des mouvements, ou la détection d'objets abandonnés.

Avec ces évolutions, le système pourrait répondre aux besoins de diverses industries et offrir une solution complète et polyvalente.